

ChartQAPro Evidence-First Reasoning

Technical Analysis of a Reproducible Multimodal Benchmarking Pipeline

Code-centric analysis of dataset integrity, prompt design,
multimodal inference, execution optimization, provenance, and evaluation

Publication-oriented technical report

Contents

Scope and Evidence Policy	1
1 Abstract	1
2 Introduction	2
2.1 Problem setting	2
2.2 Motivation for an evidence bottleneck	2
2.3 Scope of this report	2
3 Project Overview	2
4 Source Structure and Traceability	3
5 Technical Foundations	4
5.1 Multimodal conditional generation	4
5.2 Quantization	4
5.3 Batched decoder-only generation	4
5.4 Evidence as an information bottleneck	5
5.5 Paired comparison	5
6 Data and Input Processing	5
6.1 Dataset acquisition and archival strategy	5
6.2 Schema validation and normalization	6
6.3 Question semantics	6
6.4 Image handling and provenance	6
6.5 Dataset audit	7
7 Experimental Protocol and Prompt Design	7
7.1 Frozen configuration	7
7.2 Direct prompting	7
7.3 Evidence-First evidence stage	8
7.4 EFR answer stage	8
8 Output Parsing and Answer Semantics	8
9 Model Adapters and Inference Mechanics	9
9.1 Model loading	9
9.2 Padding and generation correctness	9
9.3 Batched multimodal generation	9
9.4 Out-of-memory isolation and fallback	9
9.5 Single-model residency	10
10 Resume-Safe Prediction Persistence	10
10.1 JSONL representation	10
10.2 Local spool and Drive checkpointing	10
10.3 Protocol-aware resume semantics	10
10.4 Secondary subset	11

11 Execution Optimization Layer	11
11.1 Design principle	11
11.2 Adaptive batch calibration	11
11.3 Representative probe selection	11
11.4 Deterministic image-size bucketing	11
11.5 Independent EFR text micro-batching	12
11.6 Reduced synchronization and allocator churn	12
11.7 Performance measurement hygiene	12
12 Primary and Secondary Inference Suites	12
12.1 Primary Qwen2.5-VL suite	12
12.2 Secondary SmolVLM2 suite	13
13 Evaluation Contract and Scoring	13
13.1 Official evaluator provenance	13
13.2 Evaluator contract test	13
13.3 Coverage validation	13
13.4 Official score computation	14
14 Paired Analysis and Bootstrap Procedure	14
15 Implementation Quality and Design Rationale	14
15.1 What the implementation gets particularly right	14
15.2 Why the execution optimizations are scientifically meaningful	15
15.3 Use of comments as executable methodology	15
16 Computational Considerations	15
16.1 Time complexity of core preprocessing	15
16.2 Batch calibration	16
16.3 Recursive OOM splitting	16
16.4 Memory behavior	16
17 Reproducibility and Reliability	16
18 Forensic Audit of Corrections Embedded in the Artifact	17
19 Limitations and Technical Considerations	17
19.1 Benchmark status	17
19.2 Repository evaluator choice	18
19.3 Fallback generation and protocol purity	18
19.4 Notebook global state	18
19.5 Static performance score	18
19.6 External environment dependence	18
20 Overall Technical Assessment	18
21 Conclusion	19
Source Provenance	19

Scope and Evidence Policy

This report is derived from the supplied `notebook(2).ipynb` and the matching `notebook(2).py`. The notebook contains 58 cells: 35 code cells and 23 Markdown cells. The exported Python source contains 7,731 lines, and its executable code is equivalent to the concatenation of the notebook’s code cells. The analysis therefore treats the notebook Markdown as methodological documentation and the Python source as the consolidated implementation.

The report explains what the implementation actually does, distinguishes implementation behavior from theoretical interpretation, and avoids claims that require execution evidence not present in the supplied artifact. In particular, the source explicitly states that it contains a partial T4 execution but no completed end-to-end benchmark against the pre-optimization implementation. Accordingly, this report does not present model scores, speedup figures, statistical conclusions, or visual results. The notebook contains optional plotting and result-export routines, but those are intentionally excluded from this report in accordance with the requested no-figure, no-results scope.

For traceability, major sections below cite the consolidated Python line ranges conceptually. These anchors are intended to connect prose to the implementation without reproducing the source in full.

1 Abstract

The supplied project implements a research-oriented multimodal visual question answering benchmark on the ChartQAPro test split, with a central experimental comparison between direct chart question answering and an *Evidence-First Reasoning* (EFR) pipeline. In the Direct condition, a vision-language model receives a chart image and a benchmark-specific question and is instructed to emit only the final answer. In the EFR condition, the same model first extracts a compact, machine-readable evidence object from the chart and question; a second text-only generation stage then answers the question using only that evidence. The deliberate removal of the chart image from the second stage makes the extracted evidence an explicit information bottleneck rather than a second opportunity to reread the chart.

The implementation is substantially more than an inference script. It constructs an integrity-preserving dataset acquisition layer, normalizes heterogeneous serialized values, validates real chart-image decodability, maintains both provenance-preserving and fast decoded-image caches, fingerprints the experimental protocol, archives incompatible prediction files, supports resumable JSONL prediction storage, and performs strict evaluator-contract checks before scoring. It also provides adaptive multimodal batch calibration, length-aware text-stage micro-batching, CUDA-synchronized timing helpers, optional performance profiling, and a conservative out-of-memory recovery mechanism. A secondary real-data comparison uses SmolVLM2 on a deterministic stratified subset, while the primary benchmark uses Qwen2.5-VL-3B-Instruct on the full test split.

Scientifically, the implementation is designed to test a narrow methodological question: whether inserting an explicit evidence-extraction stage changes benchmark behavior relative to direct multimodal answering, without changing the underlying dataset, model identity, generation determinism, answer normalization contract, or official scoring path. The strongest contribution of the code is therefore not any single neural architecture; it is the construction of an auditable experimental harness in which data provenance, protocol identity, inference semantics, persistence,

and scoring are treated as first-class research objects.

2 Introduction

2.1 Problem setting

Chart question answering requires a model to combine visual perception with language understanding and, in many cases, lightweight arithmetic or relational reasoning over chart elements. The supplied project studies this setting using ChartQAPro and asks whether an explicit intermediate evidence representation can improve the behavior of a multimodal model relative to direct answer generation.

The distinction is operationally precise. Direct QA maps a chart image and a question directly to an answer. EFR decomposes the same task into two stages: multimodal evidence extraction and text-only answer synthesis. The first stage is responsible for identifying observable chart facts and any necessary calculations; the second stage is forbidden from accessing the chart and can only consume the evidence JSON produced by stage one. This design isolates one specific intervention: the introduction of a constrained intermediate representation.

2.2 Motivation for an evidence bottleneck

The evidence bottleneck is important because a superficially similar two-pass prompt could simply give a model a longer multimodal context. That is not what this implementation does. The EFR answer prompt contains the question and serialized evidence only. It does not contain the chart image. Therefore any information available to stage two must have passed through stage one.

At the implementation level, the experiment is held deterministic through `TEMPERATURE = 0.0` and `DO_SAMPLE = False`, with fixed generation budgets for Direct, EFR evidence, and EFR answer stages. The protocol is additionally hashed so that prediction files produced under an incompatible configuration are not silently reused.

2.3 Scope of this report

The report covers the complete computational lifecycle: environment setup, dataset acquisition and verification, schema normalization, prompt construction, output parsing, multimodal model loading, batched generation, resume-safe persistence, calibration, primary and secondary runners, evaluator integration, paired analysis, bootstrap estimation, and final artifact packaging. It also examines explicit safeguards and implementation limitations. Optional visualization routines are discussed only as code components; no figures or result displays are reproduced.

3 Project Overview

The system can be understood as a sequence of controlled transformations:

official dataset source → verified local archive → normalized working table
→ protocol-controlled prompts → deterministic multimodal generation
→ normalized predictions → coverage-validated evaluator input
→ paired analysis artifacts.

The principal control points are listed in Table 1.

Table 1: Major architectural stages and their responsibilities.

Stage	Core components	Primary responsibility
Configuration	Experiment constants, <code>EXPERIMENT_LOCK</code> , protocol hash	Freeze scientific and execution parameters before inference
Environment	Dependency checks, runtime introspection	Record compatible software and hardware state
Dataset	Download, archive, SHA-256, normalization, image cache	Make the test data reproducible and locally reusable
Prompting	Direct/EFR templates and renderers	Encode benchmark-specific answer semantics
Inference	Model adapters, quantization, batching, OOM recovery	Execute deterministic multimodal and text-only generation
Persistence	JSONL writer, checkpoints, resume state	Preserve progress and enable interruption recovery
Evaluation	Coverage checks, evaluator contract, scoring	Prevent incomplete or malformed predictions from being scored
Analysis	Pairing, bootstrap, audit artifacts	Quantify condition differences without breaking source scoring semantics

A key architectural property is that the scientific protocol and the execution optimization layer are conceptually separated. The optimization layer changes scheduling, batching, image materialization, and persistence mechanics while retaining the same dataset rows, model identifiers, prompt texts, generation budgets, answer normalization, and evaluator route.

4 Source Structure and Traceability

The project is distributed as a notebook because execution in Google Colab is part of its operational context, but the exported `.py` file provides a linear representation of the same code. The line inventory reveals several distinct regions.

Table 2: Traceability map from implementation regions to report sections.

Source range	Topic	Report interpretation
1–92	Configuration	Experiment identity, model IDs, token budgets, T4 limits, and protocol lock
98–327	Environment	Drive layout, dependency management, runtime and GPU provenance
330–430	Instrumentation	CUDA synchronization, memory probes, and optional cell timing
501–618	Dataset archive	Download, checksum verification, persistent archive, runtime extraction
621–1084	Dataset audit	Normalization, image decoding, deterministic IDs, caching, schema checks
1053–1090	Evaluator provenance	Local copy and SHA-256 fingerprint of the repository evaluator
1092–1438	Prompting and parsing	Direct/EFR prompts, structured evidence JSON, output normalization
1440–1861	Model adapters	Quantization, loading, padding, generation, and OOM isolation
1870–2279	Persistence	JSONL writer, resume rules, stale-file archival, secondary subset
2550–3390	Optimized runner	Adaptive batching, image bucketing, EFR micro-batching, profiling, A/B harness

Source range	Topic	Report interpretation
3871–5342	Primary/secondary suites	Safe loaders, Qwen primary run, SmolVLM2 secondary run
5343–6115	Evaluation	Coverage, evaluator contract, official scoring, type-wise outputs
6118–6257	Paired analysis	Per-example pairing and seeded bootstrap confidence interval
6261 onward	Reporting and packaging	Optional figures, integrity checks, audit reports, performance status, packaging

One important implementation detail is that the file contains duplicate definitions of several functions. In Python, the later definition replaces the earlier binding in the active global namespace. The notebook uses this property deliberately in some correction cells: later definitions of the safe model loader and generation helpers supersede earlier implementations before the actual primary and secondary suites are launched. The report therefore distinguishes *conceptual function families* from the exact final binding that is active when a runner executes.

5 Technical Foundations

5.1 Multimodal conditional generation

At a high level, the model is used as a conditional generator. Given an image I , a textual prompt q , and model parameters θ , generation can be viewed abstractly as

$$\hat{y} = \text{Generate}_{\theta}(I, q), \quad (1)$$

where $\hat{y}$ is a token sequence decoded into text. The code does not implement a neural network architecture from first principles; instead it adapts pretrained vision-language models through the Hugging Face Transformers processor and generation interfaces.

The experiment uses deterministic autoregressive generation. Because sampling is disabled, the stochastic temperature parameter is not active in the normal path. The code nevertheless records temperature and sampling flags in the experiment lock so that a later protocol change would alter the protocol identity.

5.2 Quantization

On CUDA-enabled runtimes, the model is loaded with 4-bit NormalFloat4 quantization (NF4) and double quantization through `bitsandbytes`. The compute dtype is FP16. Conceptually, quantization maps high-precision weight representations to lower-precision storage while retaining a higher-precision compute path for the matrix operations in the quantized kernels. The report does not assume a universal accuracy effect; the supplied code uses quantization as a memory-management choice for T4 execution.

5.3 Batched decoder-only generation

Batched autoregressive generation requires careful handling of padding. The code forces left padding and checks for an available pad token. After generation, it trims the output at the common padded input width:

$$L_{\text{prompt}} = \text{shape}(\text{input_ids})_{-1}, \quad (2)$$

then keeps

$$Y_{\text{new}} = Y_{\text{out}}[:, L_{\text{prompt}}:]. \quad (3)$$

This is a batch-safe formulation under the left-padding convention used by the runner.

5.4 Evidence as an information bottleneck

EFR introduces an intermediate representation

$$E = f_{\theta}(I, q), \quad (4)$$

where E is a structured evidence object, and then computes

$$\hat{y}_{\text{EFR}} = g_{\theta}(q, E). \quad (5)$$

The implementation deliberately excludes I from the second stage. This gives the experiment a clean conceptual distinction from prompt chaining in which both stages still observe the original multimodal input.

5.5 Paired comparison

The Direct and EFR conditions are paired by deterministic `sample_id`. For each sample i , the code records two condition-specific scores, s_i^D and s_i^E , and forms

$$d_i = s_i^E - s_i^D. \quad (6)$$

The mean paired difference is

$$\bar{d} = \frac{1}{N} \sum_{i=1}^N d_i. \quad (7)$$

The code then estimates a percentile bootstrap interval for $\bar{d}$ using seeded resampling of the paired differences. Importantly, the report treats this as an analysis procedure encoded in the source, not as evidence that a statistically meaningful effect was observed, because no completed benchmark output is supplied as part of the artifact.

6 Data and Input Processing

6.1 Dataset acquisition and archival strategy

The dataset layer begins with the official ChartQAPro test parquet. The notebook stores the parquet inside a Drive ZIP archive without recompressing the parquet contents. The archive is accompanied by a manifest containing the dataset identifier, relative parquet path, SHA-256 hash of the parquet member, archive size, and a creation timestamp used for provenance.

The download path is Drive-first. If the archive and manifest exist and the stored size and member hash match, the archive is reused. Otherwise the parquet is downloaded from the configured Hugging Face resolution URL, hashed, and written into a ZIP container. The verified member is then extracted into a local `/content` directory for fast runtime access. This design separates persistent provenance storage from ephemeral high-speed computation.

The integrity condition can be expressed as

$$V_{\text{archive}} = \mathbf{1}[\text{size}(A) = s_m \wedge \text{SHA256}(A_m) = h_m], \quad (8)$$

where A is the archive, A_m is its parquet member, and (s_m, h_m) are the manifest values. The code uses the result of this Boolean conjunction to determine whether the persistent artifact may be reused.

6.2 Schema validation and normalization

ChartQAPro stores several fields in sequence-valued or serialized representations. The implementation therefore builds a normalization layer rather than assuming that pandas will return ordinary Python lists. The conversion sequence is approximately

```
raw value → to_python → normalize_serialized
           → as_string_list → task-specific field normalization.
```

`to_python` recursively converts NumPy arrays and scalar wrappers into native Python objects, converts pandas Series into lists, recursively visits lists and dictionaries, and changes NaN floats into None. `parse_listish_text` then recognizes bracketed strings and attempts safe parsing through `ast.literal_eval`, with JSON decoding as a secondary path.

This is not a cosmetic cleanup. The evaluator expects **Answer** and **Year** to remain lists. The prompt builders also require predictable string representations. The audit cell therefore asserts that every normalized question is a non-empty string and that answer and year fields are non-empty lists. A row-count equality assertion then checks that the normalized working table has the same number of rows as the source dataframe.

6.3 Question semantics

The function `question_for_prompt` contains a deliberate task-specific transformation. For a conversational question whose field is represented as multiple turns, all but the last item are emitted as conversation history, followed by an explicit final question marker. For other question types, a single item is used directly and multiple items are joined line-by-line.

This transformation is especially important because the final question is the target of the benchmark interaction, while the earlier turns provide context. Treating a conversational example as an unordered list would lose the distinction between history and target.

6.4 Image handling and provenance

The image layer handles bytes, byte arrays, memory views, NumPy arrays, nested dictionaries, data-URI strings, and filesystem references. `decode_chart_image` opens the encoded bytes through Pillow, forces actual image loading, records the source format, and converts the image to RGB.

Two caches are maintained. The first preserves the original encoded bytes and therefore supports provenance and content hashing. The second stores the already-decoded RGB image as a lossless PNG in local runtime storage. The latter is a performance cache: it avoids repeating JPEG/WEBP decode work and the RGB conversion on every inference batch. The source explicitly states that this cache does not alter model-side resize parameters or tensor mathematics.

The deterministic sample identifier is built from the dataframe row index, normalized question, and original image bytes. In mathematical shorthand,

$$\text{sample_id}_i = \text{rowPrefix}(i) \parallel \text{SHA256}(\text{enc}(i) \parallel 0 \parallel \text{enc}(q_i) \parallel 0 \parallel b_i)_{[12]}, \quad (9)$$

where b_i denotes the original image bytes and $||$ denotes concatenation. This creates a stable local identifier without relying on question text alone.

6.5 Dataset audit

The audit records row count, column count, image decodability, duplicate question-image pairs, missingness for key fields, question-type value counts, year values, and image cache metadata. A failed image decode becomes an explicit validation failure rather than a silently skipped record. The working table then stores normalized questions, answers, years, image hashes, cache paths, dimensions, and pixel counts.

The dimension fields are later reused for deterministic batch bucketing and calibration. Importantly, the source comments make clear that dimensions are used as scheduling metadata, not as additional model inputs.

7 Experimental Protocol and Prompt Design

7.1 Frozen configuration

The experiment begins with a fixed configuration containing model IDs, dataset identity, random seed, subset size, generation budgets, padding multiple, cache format, checkpoint interval, and several performance controls. The key scientific identifiers include:

- Primary model: `Qwen/Qwen2.5-VL-3B-Instruct`.
- Secondary model: `HuggingFaceTB/SmolVLM2-2.2B-Instruct`.
- Primary conditions: Direct and Evidence-First.
- Secondary conditions: Direct and Evidence-First.
- Primary dataset: ChartQAPro test split.
- Secondary subset size: 300 rows, selected deterministically and stratified by question type.

The experiment lock is serialized into a JSON object and later combined with the prompt configuration to form a short protocol hash. This can be viewed as a content-addressed experimental identity:

$$H_{\text{protocol}} = \text{SHA256}(\text{JSONSort}(\text{protocol material}))_{[16]}. \quad (10)$$

Any prediction record that does not carry the current hash is incompatible with the active protocol.

7.2 Direct prompting

The Direct prompt family shares strict evidence constraints: the model is told to use only information supported by the chart and supplied question, to return **unanswerable** when the chart is insufficient, and to emit only the final answer. The task-specific templates then specialize the answer format.

For Factoid and Hypothetical items, the answer is constrained to a single word, number, or short phrase, with bracketed lists permitted when multiple answers are explicitly required. Conversational prompts preserve the multi-turn history and target the final turn. Multiple Choice items require exactly one option letter, while Fact Checking requires **true** or **false**.

7.3 Evidence-First evidence stage

The EFR evidence stage asks the model to return a JSON object with exactly five top-level semantic fields: `answer_type`, `evidence`, `operations`, `support_status`, and `answer_candidate`. The evidence list is intended to contain chart-supported observations; the operations list describes necessary calculations without inventing unsupported values; the support status marks whether the chart provides enough information; and the answer candidate is tentative rather than authoritative.

The important scientific distinction is that EFR records concise, externally checkable evidence rather than hidden chain-of-thought. Nothing in the code requests or stores an unrestricted internal reasoning trace. The structured artifact is therefore designed as an observable intermediate representation.

7.4 EFR answer stage

The second EFR prompt receives the original question plus a serialized evidence JSON object. It explicitly instructs the model to use only that evidence and to return the final answer without explanation. For hypothetical questions it may recompute only operations already justified in the evidence. Because the chart is absent, the answer model cannot recover visual details that stage one failed to preserve.

This design has a useful causal interpretation. Let I be the chart, q the question, and E the extracted evidence. Direct QA computes $y = f(I, q)$, whereas EFR computes $E = f(I, q)$ followed by $y = g(q, E)$. Any stage-two error attributable to missing visual information must therefore arise from the information content of E , not from a second visual reading opportunity.

8 Output Parsing and Answer Semantics

The parsing layer is designed to remove presentation wrappers without rewriting semantics. Markdown code fences are stripped. Common prefixes such as `Final answer:` are removed. If the model returns a structured object, known answer fields such as `answer`, `final_answer`, `prediction`, or `answer_candidate` are extracted.

For Multiple Choice, wrappers such as `c)` or `option C` are normalized to the required letter. For Fact Checking, only the strings `true` and `false` are normalized. Other answer types are retained after basic presentation cleanup.

EFR JSON parsing is intentionally more conservative. A failed parse becomes an explicit object with empty evidence and operations, an insufficient support status, an empty candidate, and a parse-failed marker. That state is not silently converted into an answer. The runner marks the example as an evidence-stage failure and leaves it retryable.

The source contains direct sanity checks such as verifying that `The answer is C.` becomes `c` and that `TRUE.` becomes `true`. These tests are narrow but useful because they exercise the same normalization layer used before evaluation.

9 Model Adapters and Inference Mechanics

9.1 Model loading

The loader first clears GPU state because a new model may be switched in between suites. On CUDA, FP16 is selected as the load dtype; on CPU, FP32 is used. A single-device map places the model on GPU 0 when CUDA is present.

Qwen2.5-VL receives an explicit image-resolution range through `min_pixels` and `max_pixels`. The model loader also requests SDPA attention when enabled. If the runtime rejects the attention implementation keyword, a compatibility fallback reloads the model without that option. This is an API compatibility mechanism rather than a scientific method switch.

SmolVLM2 requires additional handling. The code explicitly imports `num2words` into the Transformers SmolVLM processing module, loads its processor, derives the authoritative tokenizer pad token, patches the model configuration before construction, and ensures that the same pad ID is reflected in the text configuration and generation configuration. The active secondary cell also normalizes local `file://` references back to filesystem paths.

9.2 Padding and generation correctness

Left padding is not treated as a hidden assumption. The safe loader verifies the processor tokenizer, checks `padding_side`, requires a pad token ID, and fails fast if the expected state is not active. For batched generation the common padded width is then used as the trimming boundary for newly generated tokens.

Generation is performed under `torch.inference_mode()`, which disables autograd bookkeeping. `use_cache=True` is included so autoregressive generation can use cached key-value states. Because sampling is disabled, calls are deterministic under the fixed model and runtime state, aside from implementation-level sources of nondeterminism that the code does not attempt to eliminate universally.

9.3 Batched multimodal generation

For Qwen, a batch is represented as a list of conversations, each containing an image item and a text item. The chat templates are applied to the batch in one operation, and visual inputs are processed through the model-specific vision utility before being passed to the processor. The processor pads token sequences to a multiple of eight, a choice motivated by T4-friendly Tensor Core dimensions.

For SmolVLM2, conversations use local image paths. The current processor API requires processor-only parameters to be routed through `processor_kwargs`; the hardened implementation follows that rule for padding and the configured video-processor parameters.

9.4 Out-of-memory isolation and fallback

The generation layer separates the normal batch call from an OOM-aware wrapper. A batch is attempted as a whole. If CUDA raises an out-of-memory exception and the batch contains more than one item, the code recursively splits the batch into two pieces and retries each half. The retry metadata records the event while preserving the successful final prediction as `status == ok`.

If a single example still fails with OOM, a smaller token budget from `OOM_FALLBACK_NEW_TOKENS` is attempted. This is a pragmatic reliability mechanism for unstable memory envelopes. It does, however, create an important distinction: a fallback generation can use a smaller maximum output budget than the nominal protocol. The source records this event in generation metadata, but the fallback itself is still intended to salvage a valid prediction rather than to change the scientific prompt or answer semantics.

9.5 Single-model residency

The suite runner loads one model, runs Direct and EFR conditions, then unloads it. This avoids paying a second model initialization within the same model suite. The optimization is execution-level: it does not alter model parameters or prompt content.

10 Resume-Safe Prediction Persistence

10.1 JSONL representation

Each prediction is serialized as one JSON object per line. The record contains the sample ID, row metadata, model ID, condition, normalized ground-truth fields, image hashes and cache paths, protocol hash, timestamps, status, prompts, raw outputs, normalized prediction, generation metadata, and GPU memory information when available.

This record structure is intentionally richer than the minimum evaluator input. The extra fields make the prediction artifact auditable without changing the later conversion into the official evaluator format.

10.2 Local spool and Drive checkpointing

When the authoritative path lies on Google Drive, `JsonlWriter` writes to a local `/content` spool and periodically copies only the new byte range to Drive. The writer does not repeatedly synchronize the full file. The copy interval is defined by `CHECKPOINT_BATCHES`; within the local file, the writer uses a buffered handle and periodic flush/checkpoint operations.

The incremental persistence rule can be written as

$$\Delta_t = \max(0, |L_t| - O_t), \quad (11)$$

where $|L_t|$ is the current local spool size and O_t is the number of bytes already persisted to the authoritative destination. Only the byte range beginning at O_t is copied at checkpoint t .

10.3 Protocol-aware resume semantics

A record is considered completed only when its protocol hash matches the active protocol and its status is `ok`. Existing files are scanned once. If records from a different protocol or malformed rows are found, the file is renamed to a stale archive rather than merged into the active run. This prevents an old result from satisfying current coverage checks merely because it has a matching sample ID.

The completion predicate is therefore

$$C(r) = \mathbf{1}[\text{status}(r) = \text{ok} \wedge \text{protocolHash}(r) = H_{\text{protocol}} \wedge \text{sample_id}(r) \neq \emptyset]. \quad (12)$$

10.4 Secondary subset

The secondary model is run on a deterministic stratified subset. The code partitions rows by question type, assigns an approximately even base allocation with deterministic remainder handling, samples each group with a seed offset by group index, shuffles the concatenated subset with the base seed, and uses a final fallback sample if an unusually small group leaves the subset short. This is a simple and reproducible approximation to preserving question-type diversity without attempting a full optimal stratified design.

11 Execution Optimization Layer

11.1 Design principle

The source repeatedly states that the scientific protocol should remain unchanged while execution mechanics improve. The optimization layer therefore targets operations such as image decoding, batch scheduling, device synchronization, model lifecycle, JSONL persistence, and repeated Python-level transformations.

11.2 Adaptive batch calibration

The runner does not simply assume that the largest stable batch is fastest. It identifies candidate sizes according to available GPU memory, tests each candidate from the upper bound down to one, measures synchronized elapsed time, estimates rows per second, and rejects candidates that violate a memory-headroom constraint.

For candidate batch size b with measured elapsed time t_b , throughput is

$$R_b = \frac{b}{t_b}. \quad (13)$$

The selected batch size is the stable candidate maximizing this quantity:

$$b^* = \arg \max_{b \in \mathcal{S}} R_b, \quad (14)$$

with larger b as the deterministic tie-breaker. $\mathcal{S}$ is the set of candidates satisfying the measured memory-safety conditions.

The calibration is keyed by model, GPU name, PyTorch version, stage name, maximum new tokens, protocol hash, and tuning-version identifier. Thus a batch size discovered for one runtime or stage is not blindly reused in a different setting.

11.3 Representative probe selection

Calibration probes use rows with large image footprints first. Image pixel count, derived from width times height, acts as a conservative proxy for multimodal memory demand. Question length is used as a secondary ordering key. This is a scheduling heuristic, not a change to the benchmark sample order or model inputs.

11.4 Deterministic image-size bucketing

Remaining rows are sorted by image-pixel count, question-character length, and sample ID. The purpose is to reduce shape heterogeneity inside a batch. Since the source uses the same per-row data regardless of order, this reordering is an execution choice rather than a content change.

11.5 Independent EFR text micro-batching

One of the more important refinements is decoupling the EFR answer-stage batch size from the multimodal evidence batch size. Successful evidence records accumulate in a pending queue and are sent to the text-only model in batches up to `TEXT_BATCH_SIZE`. Before generation, those tasks are sorted by prompt length and sample ID. This allows the text stage to exploit a different computational envelope from the image stage.

The resulting EFR execution is therefore:

multimodal evidence batches → successful evidence queue
→ length-aware text batches → final answers.

This is a meaningful systems optimization because the answer stage no longer inherits the potentially small batch size chosen for image processing.

11.6 Reduced synchronization and allocator churn

The code deliberately avoids calling `torch.cuda.empty_cache()` after every generation. It instead uses explicit garbage collection and cache clearing when switching models or recovering from OOM. This preserves allocator reuse. Timing helpers synchronize CUDA before and after benchmark windows so asynchronous kernel launches do not shift work into the next measurement.

11.7 Performance measurement hygiene

The optional timing layer distinguishes warm-up from steady-state runs, uses a fixed warm-up policy, reports median/mean/min/max, and synchronizes CUDA around measured calls. An optional IPython hook can collect end-to-end cell wall time and GPU allocator state, but it is disabled by default to avoid perturbing the scientific path.

The source explicitly distinguishes measured runtime performance from a static engineering score. The latter is computed from fixed dimension weights and implementation-derived subscores, but it is not an empirical speed measurement and is therefore not treated here as evidence of runtime improvement.

12 Primary and Secondary Inference Suites

12.1 Primary Qwen2.5-VL suite

The primary runner targets the full ChartQAPro test split. Before model loading, it checks required dataframe columns, sample-ID uniqueness, existing completion state, and the active protocol. The safe model loader verifies left padding and pad-token configuration. One model instance is then reused for Direct and EFR.

For Direct, each multimodal batch produces one final answer. For EFR, each multimodal batch produces an evidence JSON candidate; only successfully parsed evidence is sent to the text-only stage. Failure in the evidence stage creates a retryable failed record rather than an empty answer that could accidentally pass downstream type checks.

12.2 Secondary SmolVLM2 suite

The secondary path is intentionally not a copy of the Qwen adapter. It handles SmolVLM2-specific processor conventions, corrects the pad token configuration before model construction, normalizes local file references, and bypasses expensive generic batch probing in favor of safer model-specific execution. The code temporarily overrides selected global functions for the secondary run and restores the original bindings afterward.

This global-function replacement is pragmatic in a notebook, but it is also a software-engineering complexity. It relies on execution order and mutable global namespace state. The final runner is valid in the supplied notebook flow, yet it is less modular than a class-based or dependency-injected architecture would be.

13 Evaluation Contract and Scoring

13.1 Official evaluator provenance

The notebook downloads the exact repository evaluator source from the configured raw URL, stores it locally, and hashes the file. The downloaded implementation is imported dynamically. This provides a local provenance trail linking the score path to a specific evaluator source snapshot.

The notebook also acknowledges that the upstream repository recommends VLMEvalKit for more consistent evaluation and marks its own script as not recommended. The supplied code nevertheless retains the repository evaluator because it is the requested in-notebook scoring path. The report treats this as a methodological constraint rather than silently substituting another evaluator.

13.2 Evaluator contract test

Before accepting scores, the code creates a perfect self-match probe for every question type present in the working dataframe. The prediction is the final gold answer and the input fields use the normalized types expected by the evaluator. Each probe must receive an overall score of exactly one within a tolerance of 10^{-12} . A failure stops benchmark scoring.

This is a strong contract test because it catches serialization errors before model predictions are involved. In particular, it protects against the failure mode in which a list-valued field is accidentally turned into a NumPy-style string or a malformed JSON representation.

13.3 Coverage validation

The evaluation layer computes expected IDs, current-protocol IDs, successful IDs, missing IDs, unexpected IDs, duplicates, failed records, and incompatible records. A prediction file is eligible for scoring only if there are no missing or unexpected successful IDs, no duplicate successful records, no incompatible records, and the count of successful rows equals the expected dataset size.

The completeness condition is therefore approximately

$$\mathcal{E} = 1 \iff M = 0 \wedge U = 0 \wedge D = 0 \wedge I = 0 \wedge S = N, \quad (15)$$

where M, U, D, I denote missing, unexpected, duplicate-successful, and incompatible counts,

respectively, while S and N are successful and expected counts.

13.4 Official score computation

The code passes a correctly typed JSON list to `evaluate_predictions_chartqapro`. It then recomputes per-example scores using the repository function `relaxed_correctness_chartqapro` and verifies that the mean per-example score matches the evaluator’s reported split and overall scores within a very tight numerical tolerance.

This is an especially good traceability feature: the detailed CSV tables are not based on a separately implemented approximation of the evaluator. They are required to reproduce the same scoring semantics used for the aggregate result.

The source also records an upstream evaluator quirk concerning a local `always_use_exact_match` variable in constrained question types. The corrected notebook intentionally mirrors the actual function call used by the evaluator rather than inventing a different score rule. This is a useful example of distinguishing *repository behavior* from *what the benchmark arguably ought to do*.

14 Paired Analysis and Bootstrap Procedure

When both primary conditions are complete, the code loads current-protocol records keyed by `sample_id`, intersects them with the expected dataframe order, drops any pair whose status is not `ok`, and requires the final paired table to contain exactly the expected number of rows.

For each paired example, the difference is

$$d_i = s_i^{\text{EFR}} - s_i^{\text{Direct}}. \quad (16)$$

The analysis computes the sample mean $\bar{d}$ and uses a seeded nonparametric bootstrap. For each bootstrap replicate b , indices are sampled with replacement from $\{1, \dots, N\}$, and the replicate mean is

$$\bar{d}^{*(b)} = \frac{1}{N} \sum_{j=1}^N d_{I_j^{(b)}}. \quad (17)$$

The reported interval is the empirical 2.5th and 97.5th percentiles of the bootstrap means.

The code generates bootstrap indices in chunks to avoid allocating a potentially large $B \times N$ index matrix. This is a memory optimization with no intended change to the bootstrap definition. A fixed random seed controls the resampling sequence.

A methodological limitation should be stated explicitly: a paired bootstrap interval is an uncertainty estimate under the empirical resampling model; it is not by itself evidence that a difference is causally attributable to EFR. The scientific interpretation still depends on the benchmark design, model comparability, evaluator validity, and the actual completed results.

15 Implementation Quality and Design Rationale

15.1 What the implementation gets particularly right

The strongest engineering choice is the explicit separation of scientific configuration from execution mechanics. The protocol hash covers the experiment definition, while performance versions and caches are separately versioned. This makes it possible to optimize runtime without silently mixing prediction files from incompatible scientific configurations.

The second strong choice is treating data integrity as part of the computational experiment. The notebook does not simply load a parquet file and trust it. It verifies the persistent archive, audits the schema, validates actual image decoding, stores source hashes, and reuses cached normalization artifacts only when their provenance matches the current parquet hash and row/column signature.

A third strength is the disciplined evaluator boundary. The prediction file contains rich provenance, but the official evaluator is fed only a carefully typed projection of the records. The evaluator contract is checked before scoring, and detailed per-example scores are validated against the official aggregates.

A fourth strength is the resume model. The combination of protocol-aware completion, stale-file archival, JSONL append semantics, and local-to-Drive checkpointing is well suited to long-running Colab experiments in which interruptions are expected. The implementation treats resumability as a research reliability feature rather than a convenience script.

15.2 Why the execution optimizations are scientifically meaningful

The optimizations are not presented as algorithmic improvements to the reasoning method. They reduce incidental overhead while holding task semantics fixed. In particular, the lossless RGB cache removes repeated decoding; adaptive batching selects a runtime-appropriate throughput point; text-stage batching uses an independent batch size; left-padding is enforced consistently; and JSONL writes are buffered locally before Drive checkpoints.

The code’s careful handling of asynchronous CUDA work is also important. Without explicit synchronization, GPU kernels could continue executing after the CPU-side timing call returns, making reported wall times misleading. The helper `timed_call` therefore synchronizes before and after the timed function.

15.3 Use of comments as executable methodology

The notebook contains unusually detailed comments explaining why specific changes should not affect scientific semantics. These comments function as lightweight design documentation. Examples include the explanation that the fast image cache uses the same already-decoded RGB pixels, that token padding changes tensor shape but not the semantic token sequence, and that the processor API correction is an interface-level fix.

This is valuable for research reproducibility because the intended invariants are written close to the code that could otherwise violate them.

16 Computational Considerations

16.1 Time complexity of core preprocessing

Hashing a file of B bytes is $O(B)$, and decoding each image is approximately linear in the amount of encoded data plus the work required by the image codec. The one-pass dataset audit intentionally combines normalization, image decoding, hashing, and cache materialization to avoid a second full-dataset pass.

The normalized working table construction is $O(N)$ in the number of rows, excluding the cost of image processing. Deterministic sorting for execution bucketing is $O(N \log N)$. Building the completion set from a JSONL file is linear in the number of records.

16.2 Batch calibration

If $|\mathcal{B}|$ candidate batch sizes are probed and each probe costs approximately T_b , then the calibration cost is proportional to

$$T_{\text{cal}} = \sum_{b \in \mathcal{B}} T_b. \quad (18)$$

This overhead is justified only when amortized over a sufficiently long benchmark. The source therefore caches tuning decisions by runtime-specific keys. A limitation is that calibration itself executes real generation and can be expensive, especially for the secondary model.

16.3 Recursive OOM splitting

For a batch of size B , a worst-case successful decomposition into individual examples requires up to B successful generation calls plus failed parent attempts along a binary split tree. The precise constant depends on where OOM occurs. The algorithm is therefore intended as a recovery mechanism, not as a performance strategy.

16.4 Memory behavior

The primary memory controls are 4-bit weight quantization, bounded Qwen image resolution, configurable batch sizes, left-padded token batches, and avoidance of persistent tensor references after generation. The runner also records allocator statistics when profiling is enabled.

One subtlety is that peak reserved memory and free-memory headroom are both considered during batch calibration. This is more conservative than checking only whether a generation happened to survive one probe. The trade-off is potentially leaving some usable memory unexploited, but the source prefers stability on a fixed-memory T4.

17 Reproducibility and Reliability

The project includes several independent reproducibility mechanisms:

1. fixed random seeds for subset sampling and bootstrap resampling;
2. a protocol version and protocol hash;
3. exact package version constraints for critical dependencies;
4. recorded runtime, GPU, CUDA, and package metadata;
5. persistent dataset and evaluator SHA-256 hashes;
6. deterministic sample identifiers;
7. deterministic answer-generation settings;
8. strict prediction coverage checks;
9. explicit parser and evaluator contract tests;
10. resumable prediction files with current-protocol filtering.

These mechanisms address different failure modes. A fixed seed does not protect against a changed dataset, whereas a dataset hash does not protect against changed prompts. The protocol hash addresses configuration drift, while sample IDs make row-level pairing independent of transient dataframe order. The system is therefore strongest when these mechanisms are considered together.

The source nevertheless does not guarantee full bitwise reproducibility across every possible hardware and software combination. The runtime may use different kernel implementations,

library versions, or underlying CUDA behavior. The notebook records this runtime state but cannot force all external numerical libraries into identical execution across arbitrary environments.

18 Forensic Audit of Corrections Embedded in the Artifact

The notebook explicitly documents a set of defects or fragile behaviors found in the supplied implementation and the corrections applied.

Table 3: Source-documented corrections and their methodological significance.

Issue class	Correction	Why it matters
Dependency completeness	Add pinned <code>num2words</code> for SmolVLM2	Prevent secondary-model import/runtime failure
Serialization	Normalize NumPy/Pandas/list-like fields into Python objects	Preserve evaluator input types and prompt semantics
EFR templating	Replace generic <code>str.format()</code> with placeholder-only rendering	Prevent literal JSON braces from being interpreted as missing format fields
Prompt semantics	Use question-type-specific templates	Match benchmark answer contracts
Padding	Enforce left padding with explicit pad token validation	Make batched decoder-only generation unambiguous
Processor API	Route processor-only parameters through <code>processor_kwargs</code> where required	Remove API misuse without changing scientific prompts
SmolVLM2 configuration	Synchronize pad token configuration before model construction	Prevent invalid generation configuration
Resume safety	Archive incompatible prediction files	Avoid silent protocol contamination
Persistence	Use a local spool with periodic Drive checkpoints	Reduce remote filesystem overhead while preserving recovery

One correction deserves special emphasis: the EFR template bug could prevent evidence-first inference entirely because literal JSON braces were interpreted by generic string formatting. The replacement renderer performs targeted placeholder substitution only, and the sanity test deliberately includes JSON-like braces inside the question. This is a good example of a small software bug having direct experimental consequences.

Another important point is that the artifact removes an unused legacy production runner while keeping legacy-compatible single-example wrappers for diagnostic benchmarking. This reduces duplicate dispatchable code without changing the executed safe runner.

19 Limitations and Technical Considerations

19.1 Benchmark status

The most important limitation is evidentiary rather than algorithmic: the supplied artifact documents the benchmark harness but does not provide a completed, apples-to-apples end-to-end performance benchmark against the pre-optimization notebook. The source explicitly instructs the notebook not to invent speedups. This report follows the same rule.

19.2 Repository evaluator choice

The notebook itself notes that the ChartQAPro repository recommends VLMEvalKit for consistent and reproducible evaluation, while the local repository evaluator is marked not recommended. Retaining the repository evaluator is defensible as an artifact-specific requirement, but it means that results produced by this path should be interpreted as results under the preserved evaluator implementation rather than automatically as canonical leaderboard measurements.

19.3 Fallback generation and protocol purity

The OOM fallback can reduce `max_new_tokens`. This does not change the prompt text, but it does create a secondary generation budget for recovered cases. The code records this metadata, which is important for forensic analysis. A future publication-grade study may choose to report the prevalence of such fallbacks or treat them as a separate robustness condition.

19.4 Notebook global state

The secondary SmolVLM2 path temporarily rebinds several global functions and restores them after the run. This is practical in a notebook but makes the execution graph more order-sensitive than a conventional package architecture. A future refactor could encapsulate model-specific behavior behind explicit adapter classes so that the active implementation is selected by dependency injection rather than global rebinding.

19.5 Static performance score

The source includes a weighted static performance-engineering score. That score is a useful code-review artifact, but it is not an empirical runtime metric. Treating it as measured speed would be methodologically incorrect. The source itself makes this distinction, and this report preserves it.

19.6 External environment dependence

The notebook assumes Google Colab-style runtime capabilities, Google Drive mounting, CUDA for intended T4 execution, access to external model repositories, and the ability to download the official evaluator source. Those are legitimate deployment assumptions, but they reduce portability to an entirely offline or non-Colab environment.

20 Overall Technical Assessment

From a software and research-methodology perspective, the implementation is coherent and unusually careful about provenance and protocol integrity. Its central scientific idea is small and well isolated: compare direct multimodal answering with a genuine evidence-first information bottleneck. Around that idea, the notebook builds a robust infrastructure layer that addresses many practical failure modes that often remain implicit in research code.

The architecture is strongest where it turns hidden assumptions into explicit checks. Examples include verified dataset archives, normalized sequence-valued fields, image decodability tests, protocol hashes, left-padding preflight, structured evidence parsing, official evaluator contract tests, and exact coverage validation. These controls reduce the risk that a successful-looking benchmark run is actually scoring the wrong data, the wrong prompt schema, or a mixture of incompatible prediction files.

The optimization work is also conceptually disciplined. It does not claim to improve reasoning quality; it attempts to remove overhead from image decoding, batch selection, model lifecycle, GPU synchronization, and remote persistence. This separation is appropriate for a scientific artifact because it reduces the chance that a speed-oriented refactor silently becomes an experimental-method change.

The main weaknesses are mostly architectural rather than conceptual: the notebook accumulates several generations of definitions, relies on mutable global state, contains specialized correction cells that supersede earlier functions, and requires a Colab-like environment. These characteristics are manageable in the supplied execution flow, but a reusable research package would benefit from consolidating the final implementations into explicit modules and tests.

21 Conclusion

The supplied project implements a complete research-oriented pipeline for studying Evidence-First Reasoning in chart question answering. It combines benchmark-controlled prompting, real-data integrity checks, multimodal and text-only generation, protocol-aware persistence, runtime-specific execution optimization, official evaluator integration, paired per-example analysis, and seeded bootstrap estimation.

The Direct and EFR conditions are conceptually well separated. Direct QA maps the chart and question directly to an answer, whereas EFR requires the first stage to convert visual information into a structured evidence bottleneck before a second text-only stage can respond. The code is therefore testing a clearly identifiable methodological intervention rather than simply comparing two prompt lengths.

Equally important, the implementation demonstrates research software discipline. Dataset and evaluator provenance are recorded with hashes; incompatible prediction files are quarantined; answer normalization is tested; scoring is blocked until coverage is complete; detailed scores are required to reproduce official aggregates; and performance claims are explicitly withheld when measurement data are absent. Those practices make the artifact substantially more suitable for an academic software portfolio than a conventional one-off inference notebook.

The most defensible interpretation of the supplied code is consequently not that it proves EFR improves ChartQAPro performance. It does not contain sufficient completed results to support that conclusion. What it does provide is a carefully controlled experimental apparatus capable of testing that question with explicit safeguards against data drift, evaluator mismatch, protocol contamination, incomplete predictions, and misleading performance reporting.

Source Provenance

The following source identifiers and URLs are reproduced from the supplied implementation itself. They are included here as provenance records rather than as an external bibliography.

- Dataset identifier: `ahmed-masry/ChartQAPro`.
- Dataset artifact: `test-00000-of-00001.parquet`.
- Official repository: <https://github.com/vis-nlp/ChartQAPro>.
- Official evaluator source: https://raw.githubusercontent.com/vis-nlp/ChartQAPro/main/evaluate_predictions.py.
- Primary model: `Qwen/Qwen2.5-VL-3B-Instruct`.
- Secondary model: `HuggingFaceTB/SmolVLM2-2.2B-Instruct`.